

CloudGPT — Architecture & Infrastructure Report Generated: 05/09/2026, 13:26:58

 **Disclaimer:** Verify all pricing models, compliance scopes, and identity federation steps against official Amazon Web Services, Google Cloud, and Microsoft Azure documentation before deploying multi-cloud architectures to production environments.

Problem Analysis

Engineering teams attempting multi-cloud deployments frequently encounter friction when bridging disparate identity providers, networking constructs, and container runtimes across Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure. Without strict adherence to infrastructure-as-code principles, OpenID Connect (OIDC) authentication, and non-overlapping IP allocation, automated pipelines fail during synchronization or leave orphaned resources that inflate operational expenditure.

Root Cause

The underlying failure stems from treating multi-cloud architectures as three independent single-cloud silos rather than an integrated platform. Specifically, reliance on static long-lived credentials, mismatched CIDR address spaces, and uncoordinated state destruction scripts lead to intermittent cross-cloud routing failures, security policy drift, and incomplete teardowns that violate cloud financial management controls.

Step-by-Step Resolution

1. Establish OIDC identity federation across all three cloud providers to eliminate static secrets from the repository and continuous integration pipelines.
 - Configure GitHub Actions OIDC to assume an AWS IAM role with a scoped trust policy.
 - Set up a GCP Workload Identity Pool and Provider to allow secure service account impersonation.
 - Register an Azure App Registration with a federated credential trusting the specific repository branch.

2. Configure non-overlapping network backbones using an AWS Virtual Private Cloud (`10.10.0.0/16`), a GCP Virtual Private Cloud (`10.20.0.0/16`), and an Azure Virtual Network (`10.30.0.0/16`), interconnected via a secure site-to-site VPN mesh.
 - Apply least-privilege security groups, firewall rules, and Network Security Groups (NSGs) to restrict inter-cloud traffic.
 - Restrict public ingress exclusively to port 443 on TLS-terminated load balancers.
3. Provision private managed Kubernetes clusters across all three environments.
 - Deploy Amazon EKS in AWS, Google Kubernetes Engine (GKE) in GCP, and Azure Kubernetes Service (AKS) in Azure with private control plane endpoints.
 - Install a multi-primary service mesh with mutual TLS enforced in strict mode to secure east-west traffic between workloads.
4. Implement automated cost governance, budget alerts, and resource tagging validation scripts.
 - Enforce mandatory tag schemas (`project` , `environment` , `owner` , `cost_center` , `managed_by`) on every infrastructure resource.
 - Configure combined budget caps with automated alerting thresholds at 50%, 80%, and 100% of the daily limit.
5. Execute the verification and teardown runbooks to validate complete resource removal.
 - Run verification scans against cloud resource-listing APIs to confirm zero billable resources remain tagged with the project identifier.

Preventive Measures

Implement mandatory policy-as-code checks using Open Policy Agent and Conftest within continuous integration pipelines to automatically block pull requests containing wildcard IAM permissions, unencrypted storage volumes, or public ingress rules on unauthorized ports.

References

1. [AWS IAM OIDC Federation Documentation](#)
2. [Google Cloud Workload Identity Federation Guide](#)
3. [Azure App Registration Federated Credentials Documentation](#)

🤖 Max thinking · 439 tokens · 4.9s

